Object Oriented Programming – CS/CE 224/272

Nadia Nasir

Habib University, Fall 2025

Copy Constructors

Creating new objects using existing ones

https://www.learncpp.com/cpp-tutorial/introduction-to-the-copy-constructor/

In the first line, **f** has been initialized using the provided parameters.
In the second line, a new object is created (**fCopy**) using an **existing object** of the **same class** (**f**).

Our ctor's parameters are not suited for this.
Yet, this program will work just fine!

```
class Fraction
{
private:
    int m_num;
    int m_den;

public:
    Fraction(int num, int den)
        : m_num{ num }, m_den{ den }
    {
    }

    void print() const
    {
        std::cout << "Fraction(" << m_num << ", "
                  << m_den << ")\n";
    }
};
```

```
int main()
{
    // Calls Fraction(int, int) constructor
    Fraction f{ 5, 3 };

    // What constructor is used here?
    Fraction fCopy{ f };

    f.print();
    fCopy.print();

    return 0;
}
```

What is a Copy Constructor?

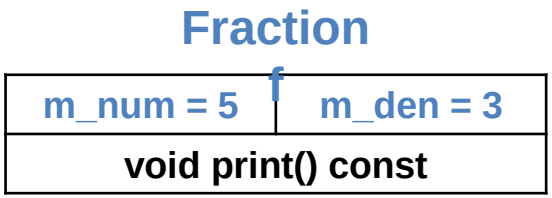
Copy constructor is a constructor that is used to initialize an object with an **existing object** of the **same type**.

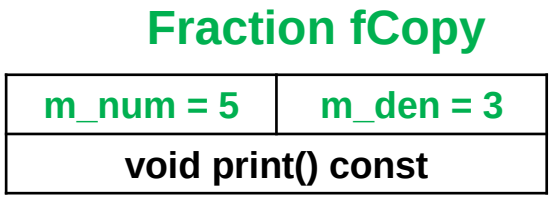
The newly created object will be **a copy of the object** passed in as the initializer.

If you do not provide a copy constructor for your classes, the compiler will provide a **public implicit copy constructor** for you.

```
Fraction fCopy{ f };
```

- **fCopy**'s attributes will be initialized using the values of the attributes of **f**.
- Note that the data of both objects is **still independent** from one another.
  - Changing **f**'s attributes **will not** impact **fCopy**'s attributes and vice versa.





This is what a copy ctor looks like.
Take note of the parameter that the copy ctor is accepting.

```
class Fraction
{
private:
    int m_num;
    int m_den;

public:
    Fraction(int num, int den)
        : m_num{ num }, m_den{ den }
    {
    }
    // Copy constructor
    Fraction(const Fraction& frac)
        : m_num{ frac.m_num }, m_den{ frac.m_den }
    {
        // just to prove it works
        std::cout << "Copy constructor called\n";
    }
};
```

```
int main()
{
    // Calls the regular constructor
    Fraction f{ 5, 3 };

    // Calls copy constructor
    Fraction fCopy{ f };

    return 0;
}
```

This is an application of ***"Access controls work on a per-class basis"***.

Where else would a copy constructor be used?

Another situation where copy ctors would be called will be when you **pass objects by value** to some function.

Pass by value means that copies of the passed parameters are created.

If your parameter is an object, then its copy will be created.

And for its copy to be created, the copy ctor will be called.

Question

Do you think that you could pass the object by value in the

```
// Calls the regular constructor
Fraction f { 5, 3 };

// Calls copy constructor
Fraction fCopy { f };
```

```
// Pass by value
Fraction(Fraction frac)
    : m_num{ frac.m_num }, m_den{ frac.m_den }
{
}
```

Not OK
Compiler error

```
// Pass by value; the copy is const
Fraction(const Fraction frac)
    : m_num{ frac.m_num }, m_den{ frac.m_den }
{
}
```

Not OK
Compiler error

```
// Pass by reference
Fraction(Fraction& frac)
    : m_num{ frac.m_num }, m_den{ frac.m_den }
{
}
```

OK (but **const** is better)

https://www.learncpp.com/cpp-tutorial/introduction-to-the-copy-constructor/

Quiz time

Question #1

In the lesson above, we noted that the parameter for a copy constructor must be a (const) reference. Why aren't we allowed to use pass by value?

Hint: Think about what happens when we pass a class type argument by value.

Hide Solution

> When we pass a class type argument by value, the copy constructor is implicitly invoked to copy the argument into the parameter.
>
> If the copy constructor used pass by value, the copy constructor would need to call itself to copy the initializer argument into the copy constructor parameter. But that call to the copy constructor would also be pass by value, so the copy constructor would be invoked again to copy the argument into the function parameter. This would lead to an infinite chain of calls to the copy constructor.

Why would you ever need to write your own copy constructor though?

Similar reason as when you would need to write your own destructor…

*Precursor to the **RULE OF THREE***

At the moment, the implicit copy ctor will be used.
Attributes of **list1** will be copied over to attributes of **list2**.
What will this result in?


```
class MyList
{
private:
    double* m_array;
    int m_length;

public:
    int getLength() const { return m_length; }

    MyList(int length)
        : m_array{ new double[length]{} },
          m_length{ length }
    {}

    ~MyList()
    {
        delete[] m_array;
    }
};
```

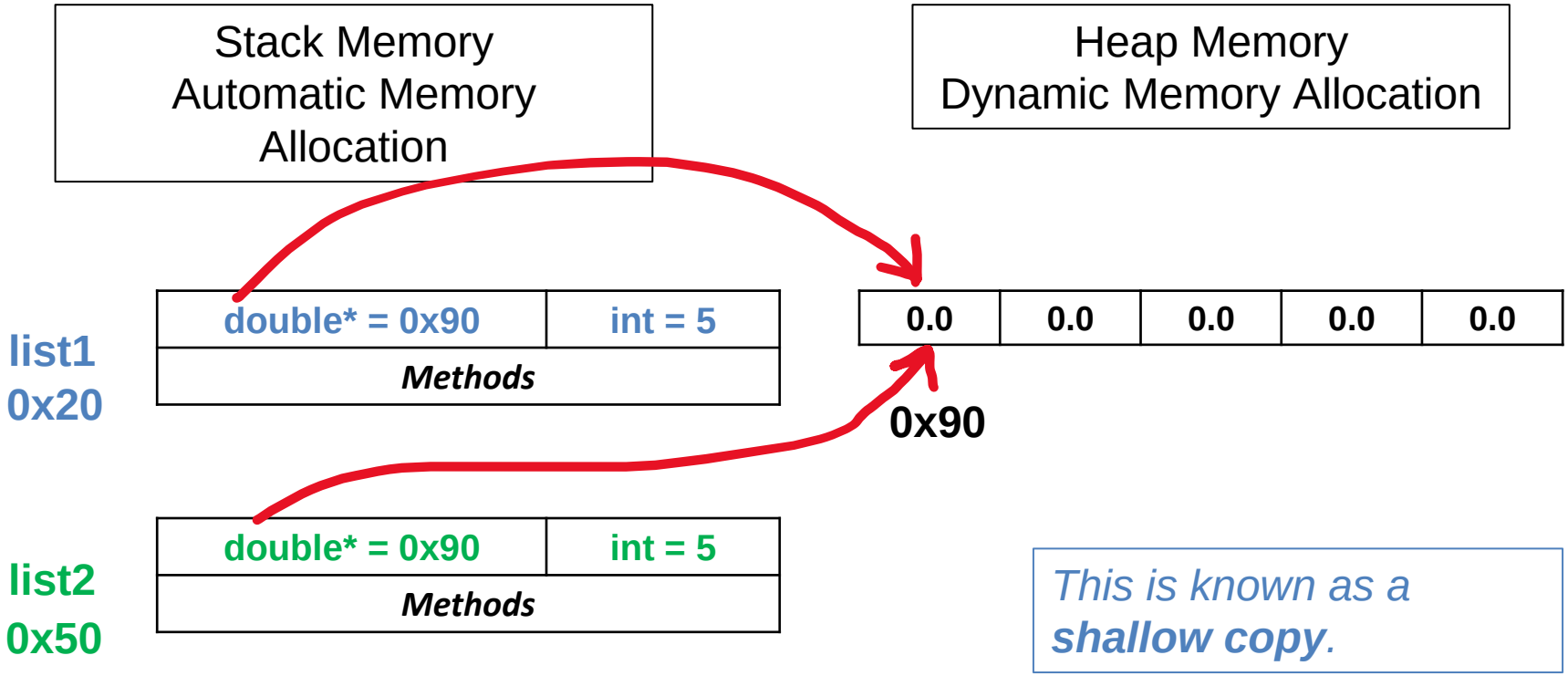
```
int main()
{
    MyList list1{ 5 };

    {
        MyList list2{ list1 };
    }

    return 0;
}
```

Btw, my goal here is to create **list2** *with its own array of* **doubles**, *but is a copy of the array of* **list1**.

At the moment, the implicit copy ctor will be used.
Attributes of **list1** will be copied over to attributes of **list2**.
What will this result in?



*This is known as a **shallow copy**.*

What will happen to the dynamic array when **list2** goes out of scope?
That array will be deallocated because of the call to the dtor for **list2**.
list1's pointer attribute will be left dangling.


```
class MyList
{
private:
    double* m_array;
    int m_length;

public:
    int getLength() const { return m_length; }

    MyList(int length)
        : m_array{ new double[length]{} },
          m_length{ length }
    {}

    ~MyList()
    {
        delete[] m_array;
    }
};
```

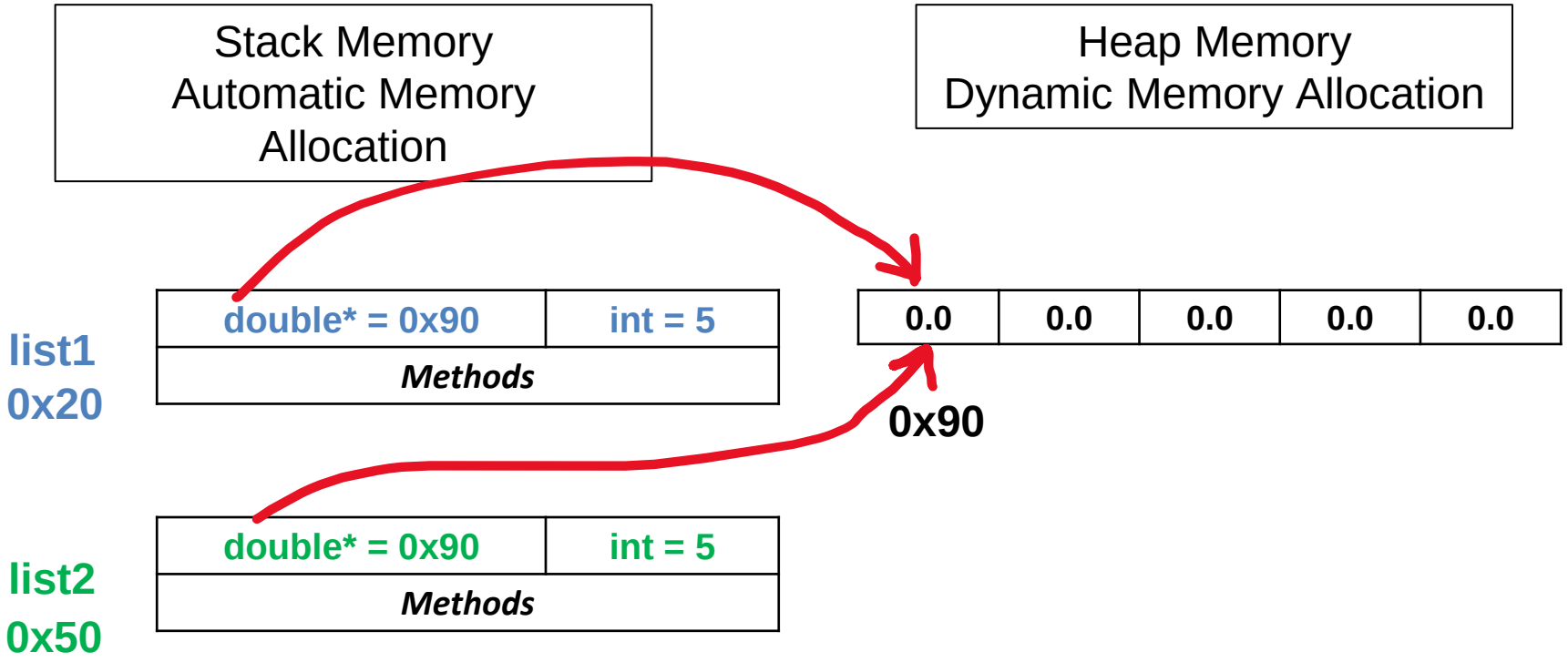
```
int main()
{
    MyList list1{ 5 };

    {
        MyList list2{ list1 };
    }

    return 0;
}
```

As **list2** is about to go out of scope, the compiler triggers a call to the dtor for **list2**.
The array on the heap gets deallocated in the dtor call for **list2**.
But **list1**'s pointer attribute is left **dangling**.

Exit animations are enabled on this slide



This is a good situation to write your own copy ctor.
Here, you allocate separate memory on the heap for the copy object.
Recall that in these type of situations, you should write **your own dtor** and **your own copy ctor**.

There is a third thing that you should also write, which we'll see later in operator overloading.

RULE OF THREE

```
class MyList
{
private:
    double* m_array;
    int m_length;

public:
    int getLength() const { return m_length; }

    MyList(int length)
        : m_array{ new double[length]{} },
          m_length{ length }
    {}

    ~MyList()
    {
        delete[] m_array;
    }
};
```

```
// Within the class
MyList(const MyList& ob)
    : m_array{ new double[ob.m_length] },
      m_length{ ob.m_length }
{
    for(int i = 0; i < ob.m_length; ++i)
    {
        m_array[i] = ob.m_array[i];
    }
}
```

I am copying the elements in the array associated with **list1** *to the array associated with* **list2** *within the copy ctor's body.*

This is known as a ***deep copy****.*

(SS) Something to keep in mind…

We mentioned that if we don't write any ctor (regular and copy), the compiler provides us with a default regular ctor and an implicit copy ctor.

If we write a regular ctor, but do not write a copy ctor, then the compiler doesn't provide a regular default ctor, but it will provide an implicit copy ctor.

However, if you don't write a regular ctor, but you do write a copy ctor, then the compiler will not provide you with an implicit copy ctor, **but it will also not provide you with a regular default ctor**.

(SS) Something to keep in mind…

Have I written a regular ctor (default or otherwise)?	Have I written a copy ctor?	What will the compiler provide me with?
No	No	Regular default and copy ctors
Yes	No	Implicit copy ctor
Yes	Yes	Nothing
No	Yes	Nothing

Note that this table is just focusing on these two types of constructors.

We are not talking about other things that the compiler provides us with.